

Mitigating CPU Spikes via State Ramping and Subnormal Prevention in C++ DSP

Bill Westlake

July 30, 2026

Abstract

Real-time digital signal processing (DSP) in C++ requires rigid adherence to mathematical continuity and hardware constraints. This paper examines two primary vectors for audio artifacts and CPU overload: mathematical discontinuities caused by instantaneous parameter changes, and hardware-level microcode traps triggered by subnormal (denormal) floating-point numbers. We present structural C++ solutions for both vectors—state ramping and hardware flag initialization—and illustrate how the Creation Station Audio Engine enforces these paradigms natively.

1 The Physics of Audio Artifacts

A digital audio signal is a continuous pressure waveform approximated by discrete temporal samples. When a user or an automation system modifies an audio parameter (such as volume gain, pan, or a filter cutoff frequency), naive implementations will apply the new value instantaneously within the audio callback.

Mathematically, an instantaneous change creates a step function (a discontinuity) in the signal’s amplitude. According to Fourier analysis, a step function is composed of an infinite series of high-frequency sine waves. When this mathematical discontinuity is pushed to the Digital-to-Analog Converter (DAC) and amplified through physical speakers, the burst of broadband high-frequency energy is perceived by the human ear as a sharp “click” or “pop.”

Therefore, maintaining acoustic integrity requires that all parameter changes over time t remain differentiable (or at least continuous). We must convert step functions into linear or exponential ramps.

2 Mathematical Smoothing and Ramping Strategies

To prevent discontinuities, DSP algorithms must interpolate from an *old_value* to a *new_value* over a predetermined duration (often the length of the current audio buffer, or a fixed time constant such as 20ms).

The most efficient algorithm for this in real-time C++ is the linear ramp, calculated once per block and incremented per sample.

```
class SmoothedParameter {
public:
    void setTargetValue(float target, int numSamplesToReachTarget) {
        targetValue = target;
        if (numSamplesToReachTarget > 0) {
            increment = (targetValue - currentValue) / numSamplesToReachTarget;
            samplesRemaining = numSamplesToReachTarget;
        } else {
            currentValue = targetValue;
            increment = 0.0f;
            samplesRemaining = 0;
        }
    }

    float getNextValue() {
        if (samplesRemaining > 0) {
            currentValue += increment;
            samplesRemaining--;
        }
        return currentValue;
    }

private:
    float currentValue = 0.0f;
    float targetValue = 0.0f;
    float increment = 0.0f;
    int samplesRemaining = 0;
};
```

By calling `getNextValue()` for every sample in the `processBlock` loop, the parameter smoothly transitions, eliminating the mathematical discontinuity and the resulting audible artifact.

3 The Subnormal (Denormal) CPU Trap

While instantaneous changes cause audible artifacts, mathematical decay can cause catastrophic CPU overload.

In infinite impulse response (IIR) filters, reverbs, and feedback delays, a signal often decays exponentially toward zero. Eventually, the signal reaches absolute silence. However, in the IEEE 754 floating-point standard, numbers extremely close to zero (typically smaller than 1.18×10^{-38} for 32-bit floats) are classified as *subnormal* or *denormal* numbers.

Standard x86 and ARM Floating Point Units (FPUs) are highly optimized for normalized numbers. When an FPU encounters a subnormal number in a multiplication or addition operation, it cannot process it in standard hardware registers. Instead, it throws a hardware exception, forcing the CPU to handle the math via microcode.

Microcode execution can be hundreds of times slower than native FPU execution. When a delay tail drops into the subnormal range, the execution time (E_t) of the audio callback violently spikes. If E_t exceeds the buffer duration (L), the audio engine drops the buffer, resulting in a system-wide stutter—ironically caused by a signal that is entirely inaudible.

4 Hardware Mitigation via FTZ and DAZ

Historically, DSP engineers solved the subnormal problem by injecting a microscopic DC offset (e.g., 10^{-25}) into the signal to prevent it from ever reaching absolute zero. However, this pollutes the signal and is computationally wasteful.

The modern architectural solution is to instruct the CPU to round subnormal numbers to absolute zero at the hardware level. This is accomplished using two CPU flags:

- **Flush-To-Zero (FTZ):** If an operation results in a subnormal number, round it to zero.
- **Denormals-Are-Zero (DAZ):** If an operation takes a subnormal number as an input, treat it as zero.

Because setting CPU flags modifies global thread state, it must be handled carefully. We utilize the Resource Acquisition Is Initialization (RAII) pattern to set the flags at the beginning of the audio callback and restore them when the callback exits.

```
class ScopedNoDenormals {
public:
    ScopedNoDenormals() {
        // Save current CPU state
```

```

        previousState = _mm_getcsr();
        // Set FTZ (bit 15) and DAZ (bit 6)
        _mm_setcsr(previousState | 0x8040);
    }

    ~ScopedNoDenormals() {
        // Restore previous CPU state
        _mm_setcsr(previousState);
    }

private:
    unsigned int previousState;
};

// Usage inside the audio graph callback:
void processGraph(float* buffer, int numSamples) {
    ScopedNoDenormals ftzDazLock;

    // All DSP processing below this line is immune
    // to subnormal microcode traps.
    executeDSP(buffer, numSamples);
}

```

5 The Creation Station Implementation

In production environments, relying on individual developers to manually implement linear ramps and FTZ locks for every plugin is a systemic liability.

The **Creation Station Audio Engine** eliminates these liabilities through architectural enforcement:

1. **Enforced Ramping:** Raw floating-point pointers are never exposed to the UI. All parameter changes pass through the engine’s proprietary automation queues, which automatically wrap targets in optimized **SmoothedValue** objects. It is architecturally impossible to introduce a step-function parameter change into the Creation Station graph.
2. **Graph-Level Hardware Locks:** The **ScopedNoDenormals** RAI lock is baked directly into the master **processBlock** of the graph executor. Whether a third-party plugin or a custom internal delay node is running, the CPU is securely locked into FTZ/DAZ mode before a single sample is processed.

By solving these mathematical and hardware edge cases at the engine infrastructure level, Creation Station guarantees that processing remains deterministic, artifact-free, and immune to hidden microcode performance traps.